

MotionPainter — SoloDeveloper Implementation

Plan Worker + MultiSampler Architecture

This document is a solo developer, step by step plan to integrate a MultiSampler into your MotionPainter project, plus worker and storage adjustments. It focuses on small, reversible changes you can make locally (with git) and push to GitHub as a backup. Follow each step in order; commit frequently.

Quick glossary

- **Core** = MotionPainter main app (FrameBuffer, renderer, composite).
- **Worker** = preprocessor.worker.js (off main thread).
- **Sampler** = MultiSampler module (CPU sampling + plugins).
- **Storage** = storage.js (IndexedDB used by worker).

One-time: GitHub backup (optional but recommended)

1. Create remote repository and push your current baseline. Minimal commands (replace placeholders):
git init git add . git commit -m "WIP: baseline before worker-sampler changes" git remote add origin https://github.com/yourname/motion-painter.git git branch -M main git push -u origin main This gives you a remote backup and ability to tag checkpoints.

Step by step plan (solo developer)

Step 0 — Prepare repo on GitHub & mirror local code (one-time)

Back up work to GitHub. See commands above.

Step 1 — Create a local feature branch

Run: git checkout -b feature/worker-multisampler-arch. Isolate changes.

Step 2 — Stabilize eviction → worker pipeline

Make sure this.frameBuffer is initialized before attaching evictionHook in main.js. Add defensive checks in FrameEvictionHook.attach().

Step 3 — Commit a checkpoint

Commit the stabilization change so you have a clean baseline to return to.

Step 4 — Create minimal MultiSampler skeleton (non-invasive)

Add src/js/sampling/MultiSampler.js with a small class: constructor, async sample(input, opts) returning a stubbed result, and plugin hooks.

Step 5 — Add a tiny topology plugin (bitmap-light)

Add `src/js/plugins/topologyPrime.basic.js` exporting a cheap analysis (edge count, bbox) for thumbnails.

Step 6 — Integrate MultiSampler into the worker (safe, feature-gated)

Instantiate the sampler in `preprocessor.worker.js` but guard calls behind an `ENABLE_SAMPLER` flag. Keep original thumbnail/phash flow intact.

Step 7 — Experimental run & metrics

Toggle `ENABLE_SAMPLER` locally to true, run `npm run dev`, ensure worker logs sampler output without crashes.

Step 8 — Store sampler output into Storage

Write mask and sample manifest artifacts via existing storage API (use consistent keys like `mask:srcHash:WxH`).

Step 9 — Adaptive queue/backpressure (worker wrapper)

In `PreprocessorWorker.js` implement `maxQueue=32` (grow to 64), low/high-water thresholds (0.4/0.9), and progressive degradation: grow queue → downsample → probabilistic skip → drop as last resort.

Step 10 — Minimal tests & telemetry

Expose a dev helper `window.getWorkerMetrics()` aggregating worker/storage metrics for tuning thresholds.

Step 11 — Next phase options

Choose GPU reducer (`OffscreenCanvas`), Blob worker for deep analysis, or sampler feature work (Wallis sieve, temporal modes).

Adaptive Queue / Backpressure: policy summary

Goal: prevent pending queue size from exceeding hard limits while preserving important frames. Policy (progressive):

1. When queue utilization rises, first increase soft capacity (grow `maxQueue` up to a hard cap, e.g., 64).

2. When utilization reaches a high threshold (e.g., 90% of hard cap), begin gentle downsampling of thumbnails.

3. If pressure continues, enable probabilistic skipping (low probability initially).

4. As a last resort when the hard cap is hit and resources are exhausted, drop lowest-priority frames (respecting pinned/high-priority frames).

Priority: frames with `meta.hasMotion`, `meta.force`, or `meta.isKeyframe` are exempted from dropping where possible.

MultiSampler: design notes

- Input-agnostic: accept bitmaps, offscreen canvases, blob references, or preloaded arrays.
- Core samplers: Random + Wallis sieve. Provide a blending parameter to mix them.
- Plugin architecture: plugins implement `run(input, options)` and return sampling hints or masks. The sampler orchestrates plugin calls and merges outputs into a manifest.
- Temporal support: sampler can optionally accept a temporal window (N thumbnails) and compute temporal weights or motion energy. Heavy temporal FFTs are deferred to a blob worker.

File layout suggestions (minimal changes)

- `src/js/sampling/MultiSampler.js` — core sampler.
- `src/js/plugins/topologyPrime.basic.js` — lightweight plugin for thumbnails.
- `src/js/core/preprocessor.worker.js` — worker: instantiate sampler, create artifacts, write to storage.
- `src/js/core/PreprocessorWorker.js` — main-thread wrapper: adaptive queue/backpressure.
- `src/js/core/storage.js` — already present; add artifact naming conventions.
- Optional: `src/js/core/blob.worker.js` for heavy analysis; `src/js/core/gpu.reducer.worker.js` for OffscreenCanvas reduction.

Practical solo-dev workflow tips

- Commit after every step with clear messages. Use feature branches for experiments.
- Use feature flags (`ENABLE_SAMPLER` etc.) so you can flip behaviour without reverting code.
- Tag checkpoints (e.g., `git tag checkpoint-01-eviction-worker`) and push tags to GitHub as your backup points.
- If something breaks, reset to last checkpoint: `git reset --hard checkpoint-01-eviction-worker` (careful—this discards local changes).

Appendix: Example commands

Create branch:

```
git checkout -b feature/worker-multisampler-arch
```

Commit:

```
git add . git commit -m "feat(sampler): add MultiSampler skeleton"
```

Push feature branch (once remote exists):

```
git push -u origin feature/worker-multisampler-arch
```

Tag a checkpoint:

```
git tag checkpoint-01-eviction-worker git push origin --tags
```